

Chapter 5: Testing

Software Testing Documentation

5.1 Functional Testing

5.1.1 Unit Testing

Unit testing focuses on validating individual components such as API controllers (e.g., AuthController, CasesController, ContractsController, AppointmentsController, AdminController), services (e.g., IAuthService, ICaseService, IContractService, IAppointmentService, IAdminService), and repositories. Each method is tested in isolation using mock dependencies to ensure that the business logic performs as expected without external interference.

5.1.2 Integration Testing

Integration testing ensures the correct interaction between interconnected components. It covers scenarios where controllers communicate with the database and services, such as:

AppointmentsController interacting with IAppointmentService and LegalMateDbContext to validate appointment scheduling and status transitions.

CasesController interacting with ICaseService to manage case life cycles and document associations.

ContractsController interacting with IContractService and Xceed.Words.NET library to process Word templates and generate contracts.

Additionally, it includes the integration of the AI model for legal queries processed by AIController via IAIService, validating API requests, handling structured responses, and ensuring robust error handling for API failures.

5.1.3 Additional Testing

System Testing:

Evaluates the behavior of the entire platform as a cohesive unit. Examples include complete user flows, such as the full journey from AuthController login to performing a legal search, generating a contract via ContractsController, or scheduling a consultation with a lawyer via AppointmentsController.

Regression Testing:

Ensures that new feature implementations or bug fixes, such as updates to the AdminController or modifications to the analytical models in AnalyticsController, do not break existing platform functionalities.

5.2 Non-Functional Testing

5.2.1 Performance Testing

Validates system response times and throughput under normal usage. Critical endpoints such as /api/Contracts/generate, /api/Cases, and /api/Appointments are tested to ensure optimal processing speed and reliable service delivery.

5.2.2 Load Testing

Examines system behavior under increasing concurrent user loads. We simulate multiple users performing concurrent requests, such as generating contracts simultaneously or querying the AnalyticsController for complex data analysis, to ensure system scalability and stability.

5.2.3 Stress Testing

Pushes the system beyond its expected limits by generating continuous, high-intensity requests, such as excessive document uploads in DocumentsController or simultaneous AI chat requests in AIController. This helps identify breaking points, memory bottlenecks, and the platform's system recovery capabilities.

5.2.4 Security Testing

Assesses how well the system protects against unauthorized access. We verify the Role-Based Access Control (RBAC) implementation across AdminController, AnalyticsController, and AppointmentsController, ensuring that sensitive legal data and Admin features are exclusively accessible by authorized roles. Password security via hashing is verified in AuthController, and endpoints related to document handling are tested for strict authorization checks.

5.2.5 Acceptance Testing

Acceptance testing ensures that the application fulfills the business and user requirements before final deployment. Typical end-to-end flows, including user registration, legal document generation, and appointment management, are tested to confirm that the system meets user expectations and is fully ready for a production environment.